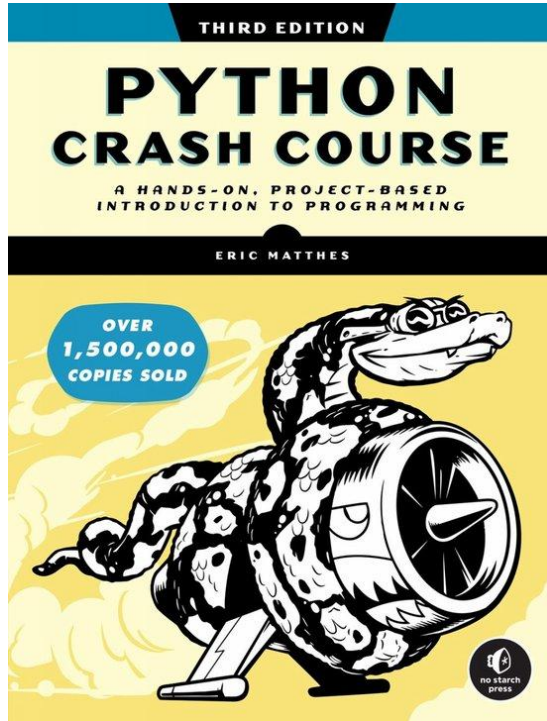


Introduction Machine Learning

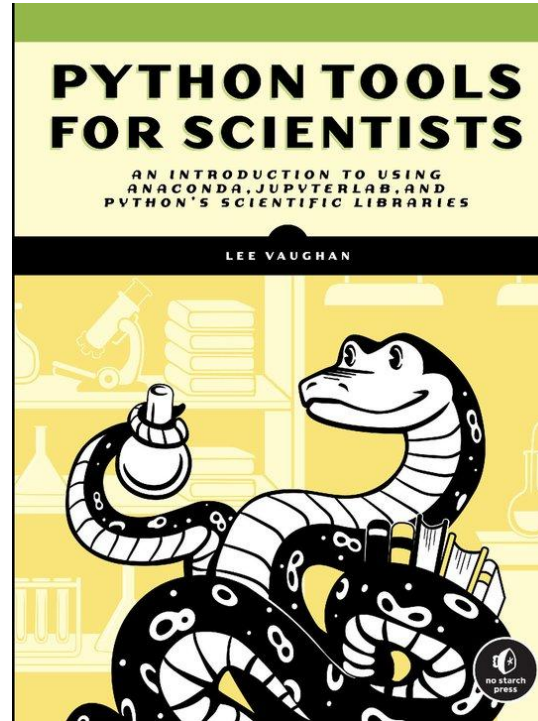
- General purpose programming language first released by Guido van Rossum in 1991
- Python has grown into the most popular language for Data Science
 - Combines capabilities of Excel, JMP, MATLAB, etc. into one package
- Used by all major tech companies (also, or especially, for AI research)
- It's open-source
 - Everybody can use (and contribute new) libraries/modules/packages
 - There's a huge community of contributors (> 400,000 packages available!)
- Scientific papers about machine learning (incl. for chemistry) very often includes Python code that you can try and use
- AI can help you a lot with Python coding (but basic knowledge is strongly recommended)

Recommended books

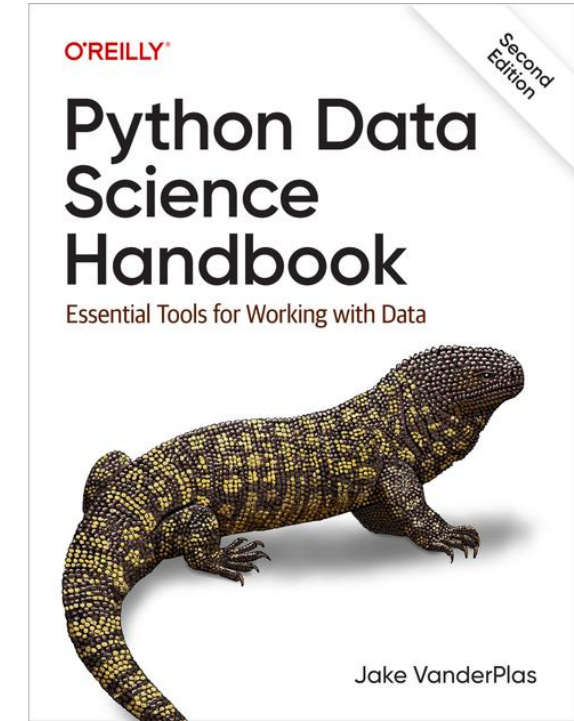


Only native Python

- Functions
- OOP
- Example projects



- Native Python
- Virtual environments
- JupyterLab
- File I/O
- Libraries (not extensive)



Extensive on:

- Numpy
- Pandas
- Matplotlib
- Scikit-Learn (ML)

Using GitHub and Google Colab in this workshop

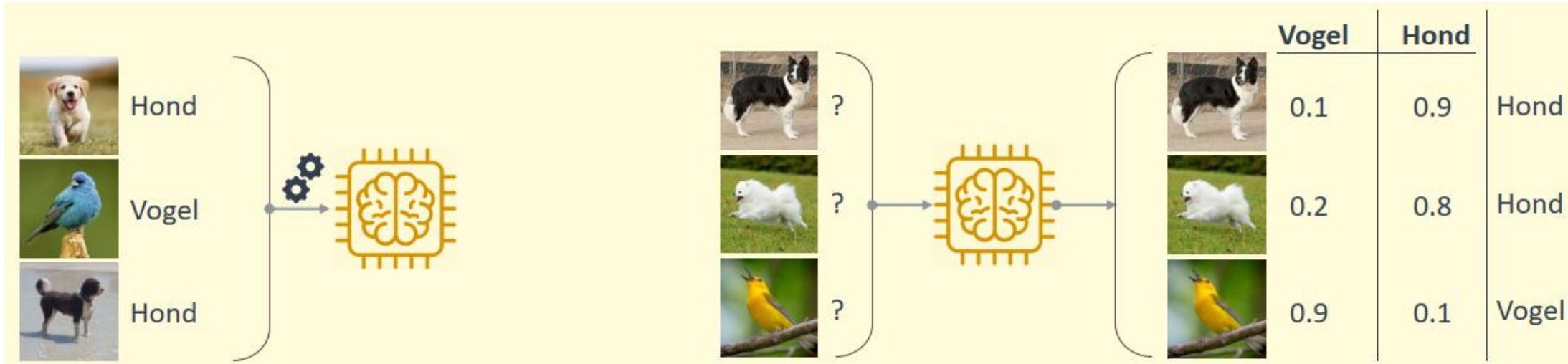
1. Go to <https://github.com/DrCOAST/coast-python>
 - Download "Auto.csv" from the "data" folder to your local work folder
2. Go to <https://colab.research.google.com/>
 - Sign in with your Google account
3. In Colab:
 - File → Open notebook
 - Select GitHub → Enter "DrCOAST/coast-python"
 - Select notebooks/COAST_Python_course_Part_1_Basics.ipynb
 - Upload "Auto.csv"

Machine Learning is everywhere nowadays: Examples

Category	Examples
Language	E-mail spam filters, chatbots, Google Translate, ChatGPT, etc.
Computer vision	Facial recognition, medical image analysis, object detection in autonomous vehicles
Finance	Fraud detection, algorithmic trading, loan risk assessment
Marketing	Personalized ads, customer segmentation, recommendation systems (<i>e.g.</i> , Netflix, Spotify)
...	...

In general, there's a lot of data available for these ML applications

Example: Machine Learning for image recognition



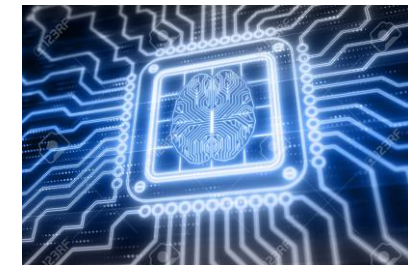
01. Train model on many images of different animal classes

02. Use model to predict the class of new images

Traditional Programming (physics-based modeling)

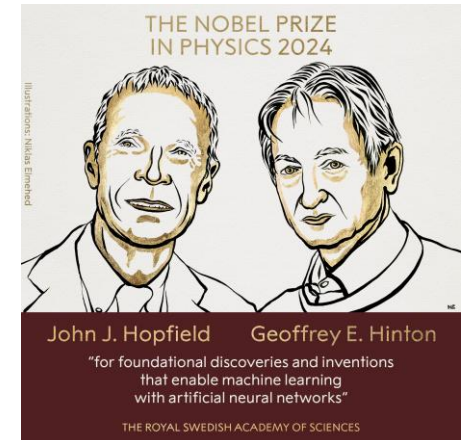
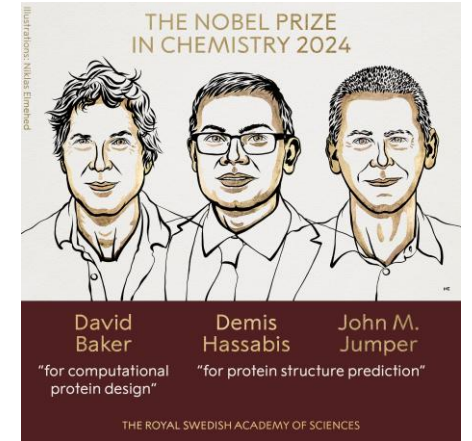
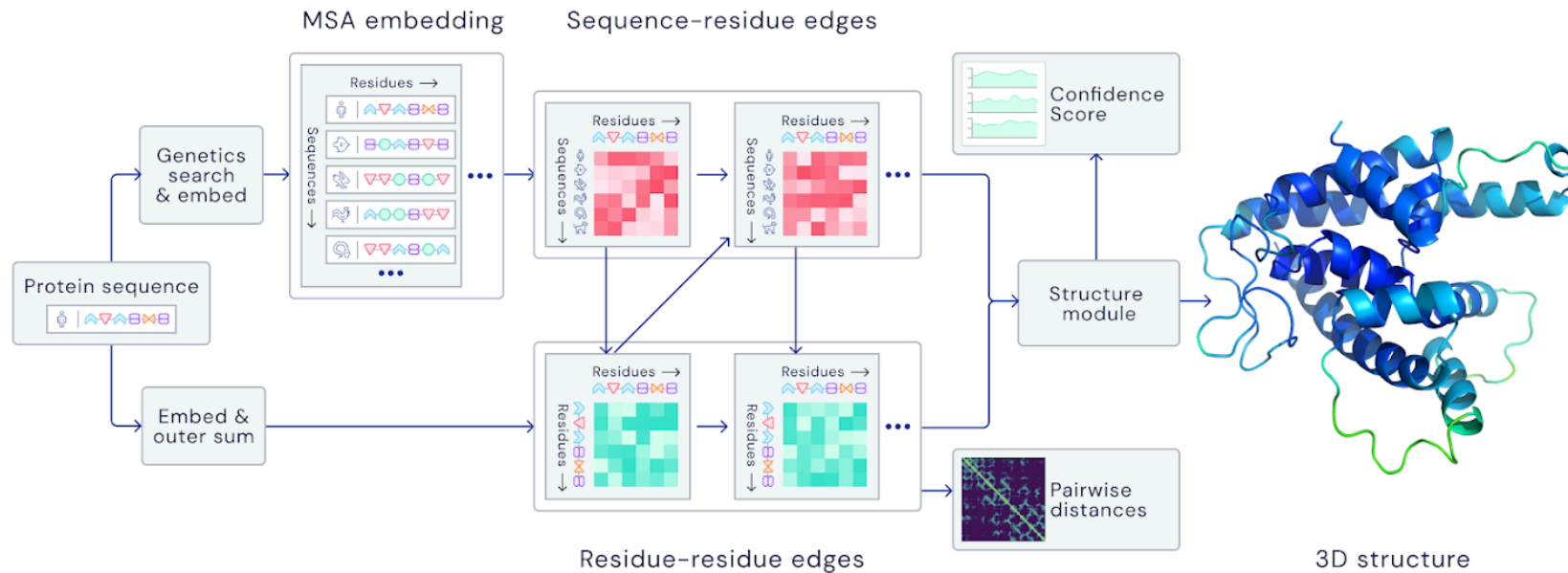


Machine Learning (data-driven modeling)



The ML Killer Application in Science (so far): AlphaFold

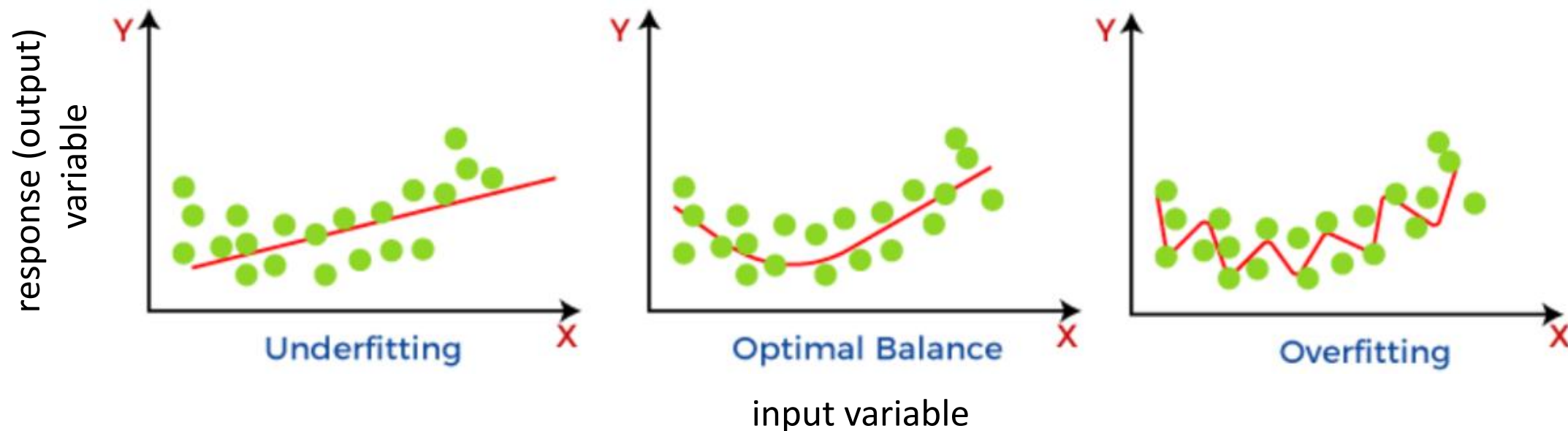
- Prediction of protein folding based on amino acid sequence
- Trained on > 200k known protein structures
- Neural network-based deep learning
- Nobel Prizes 2024 (Chemistry & Physics)



Why has ML become so popular now?

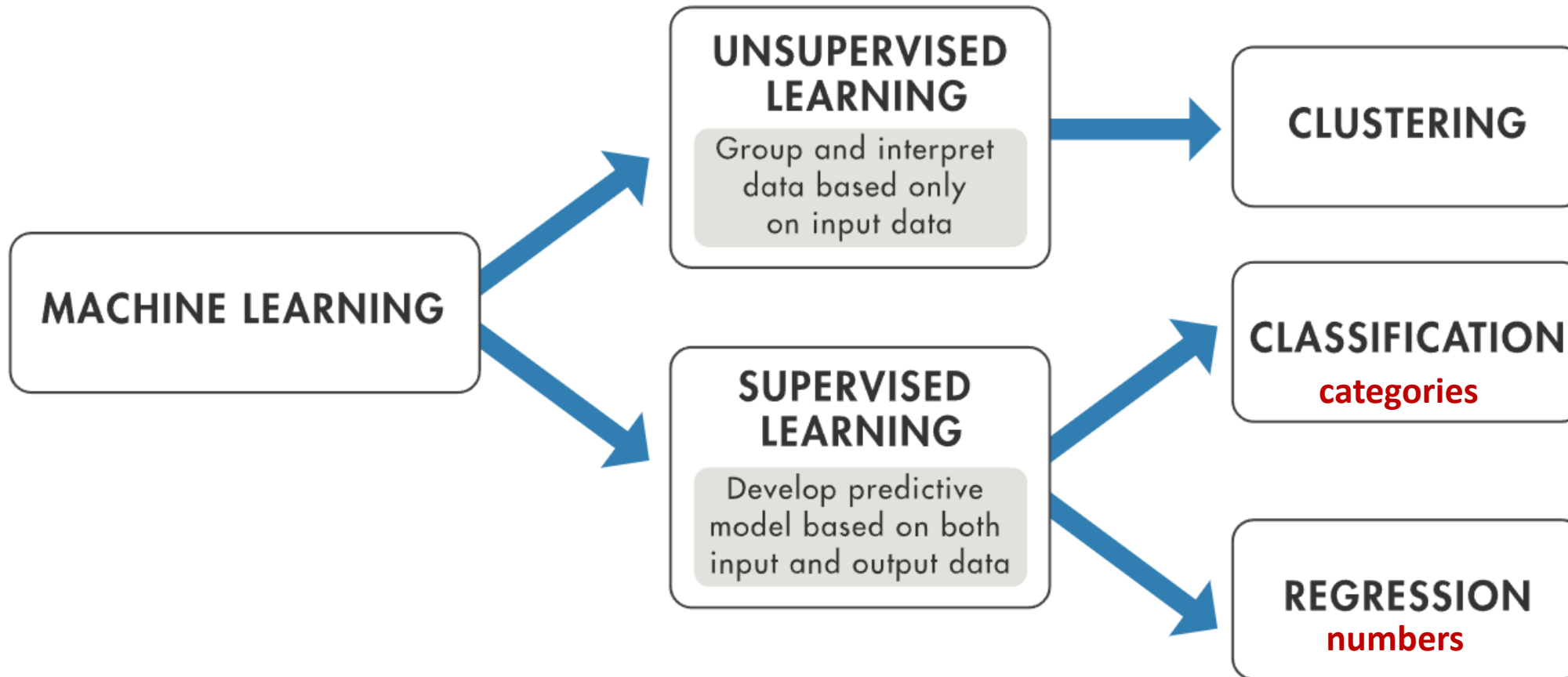
1. Availability of large datasets
2. Availability of (cheap) computing power (e.g., multi-core CPUs, GPUs)
3. Availability of open-source algorithms and easy-to-use software
(e.g., Python 😊)

Simple Univariate Regression is also Machine Learning!



Goals of ML:

- Learn relationship between input and output variables in order to make predictions for new cases (generalization)
- Improve understanding of (chemical) systems



- Multiple input variables (continuous, categorical, or combinations)
 - Examples in analytical chemistry: spectra (NMR, MS, (N)IR), chromatograms
 - Multiple sample properties
 - Representation of molecules
- Multiple response variables
 - For example, concentration and viscosity
- Non-linear relationship between input and response variables
 - More complex algorithms needed → more data needed
- (Partly) missing data
- Etc.

Matrix ($n \times p$): X -data

	x_1	...	x_p
Obs. 1	...	...	...
Obs. 2	...	...	...
Obs. ...	...	...	...
Obs. n	...	...	...

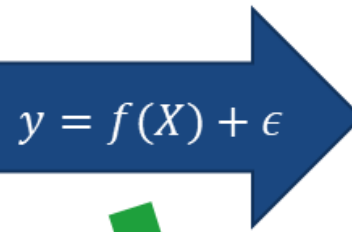
x : independent variables, (input) features, attributes

Obs.: observations, data points, (samples)

Vector or matrix: y -data

	y_1	...	y_q
Obs. 1	...	...	...
Obs. 2	...	...	...
Obs. ...	...	...	...
Obs. n	...	...	...

y : dependent variable, response, target, output, label

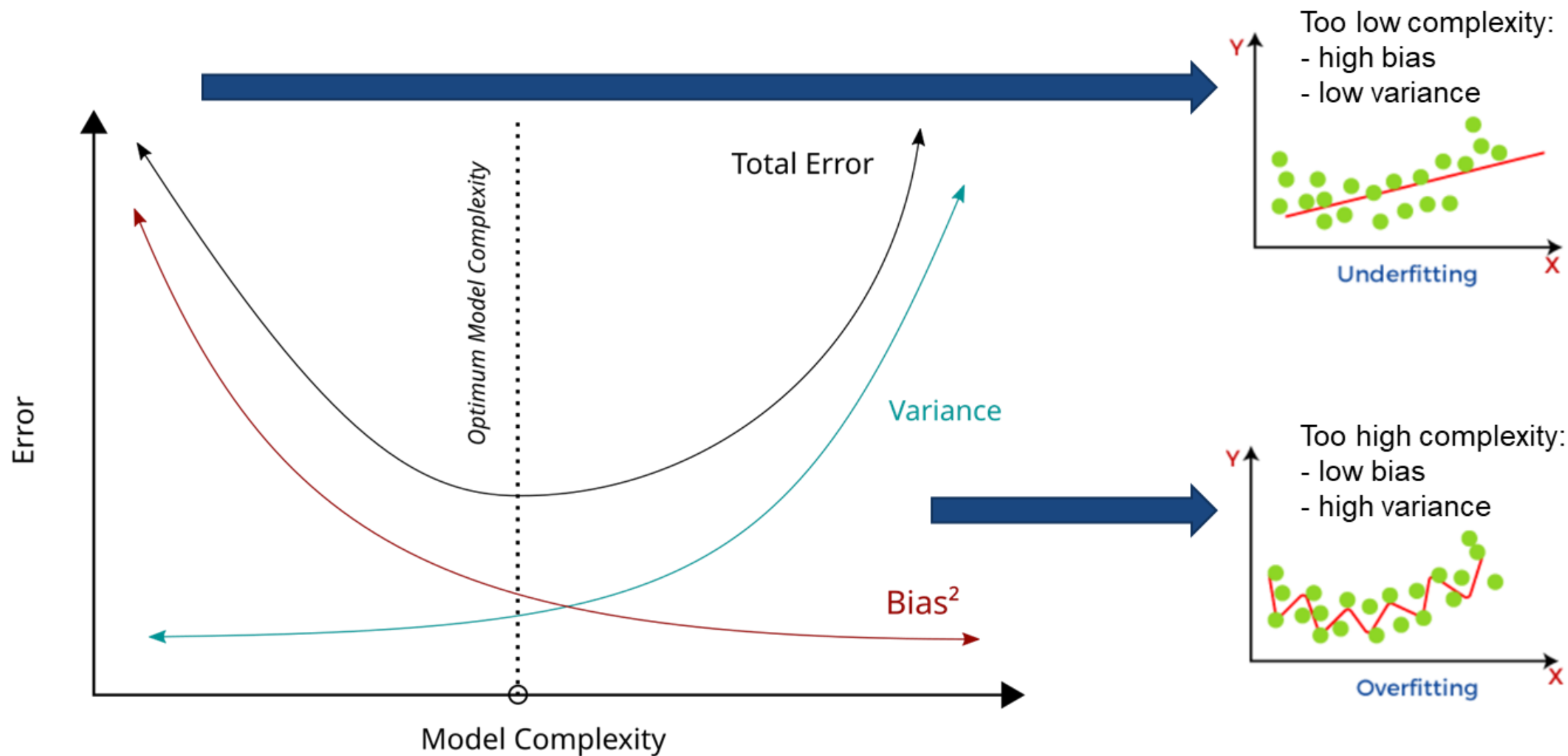

$$y = f(X) + \epsilon$$

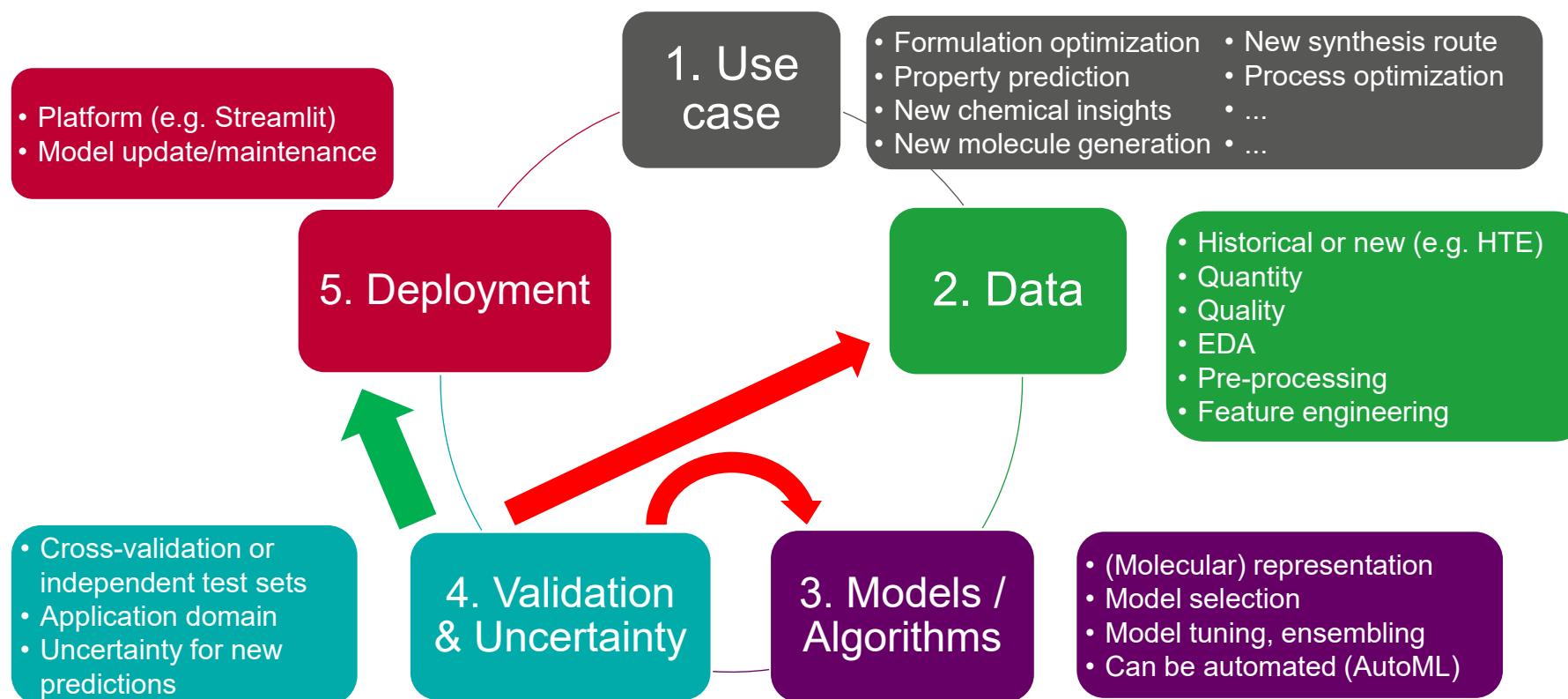


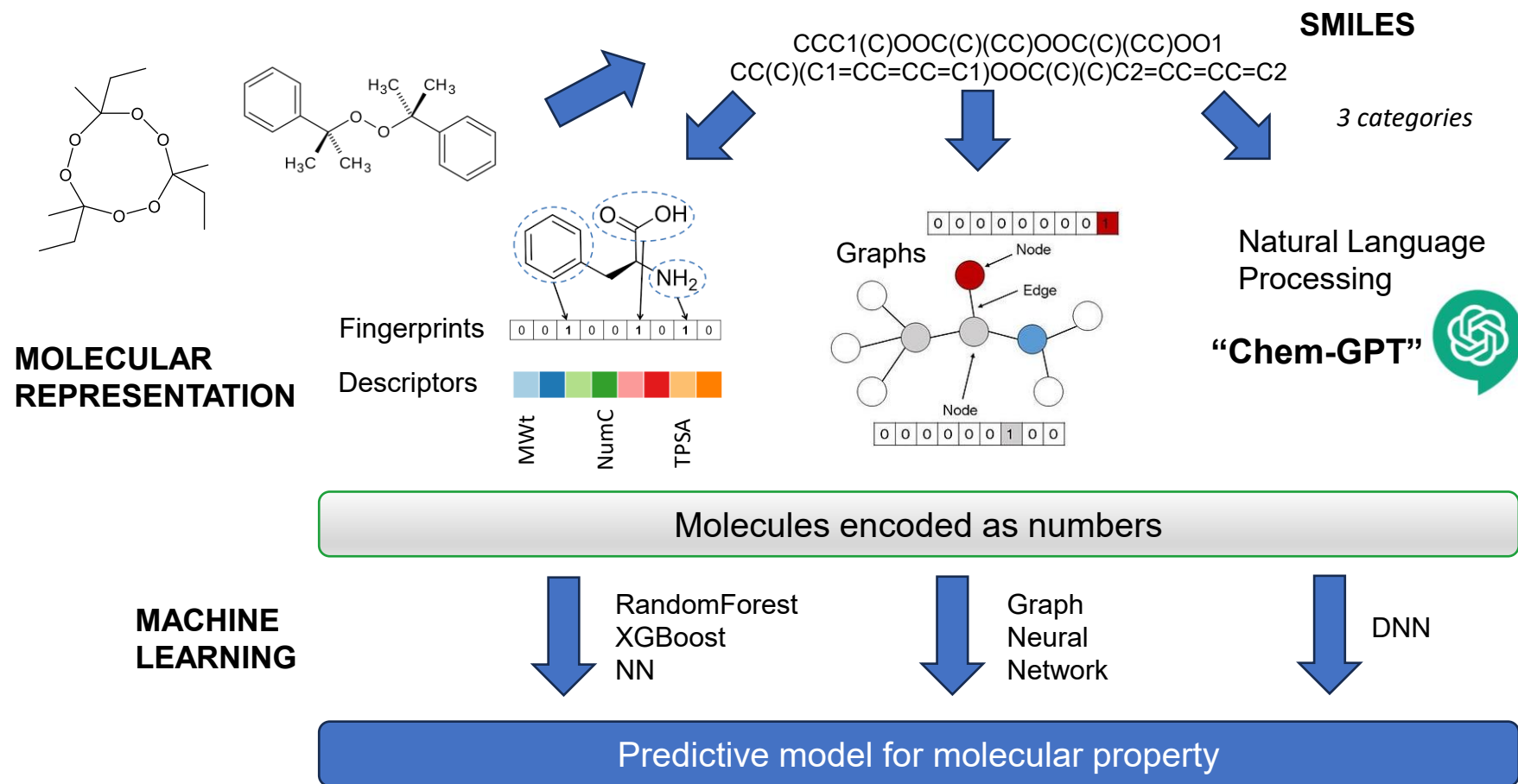
Simple Linear Regression: $y = \beta_0 + \beta_1 x_1$
Multiple Linear Regression: $y = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$

β : coefficients, parameters

Important concept: The bias – variance trade-off

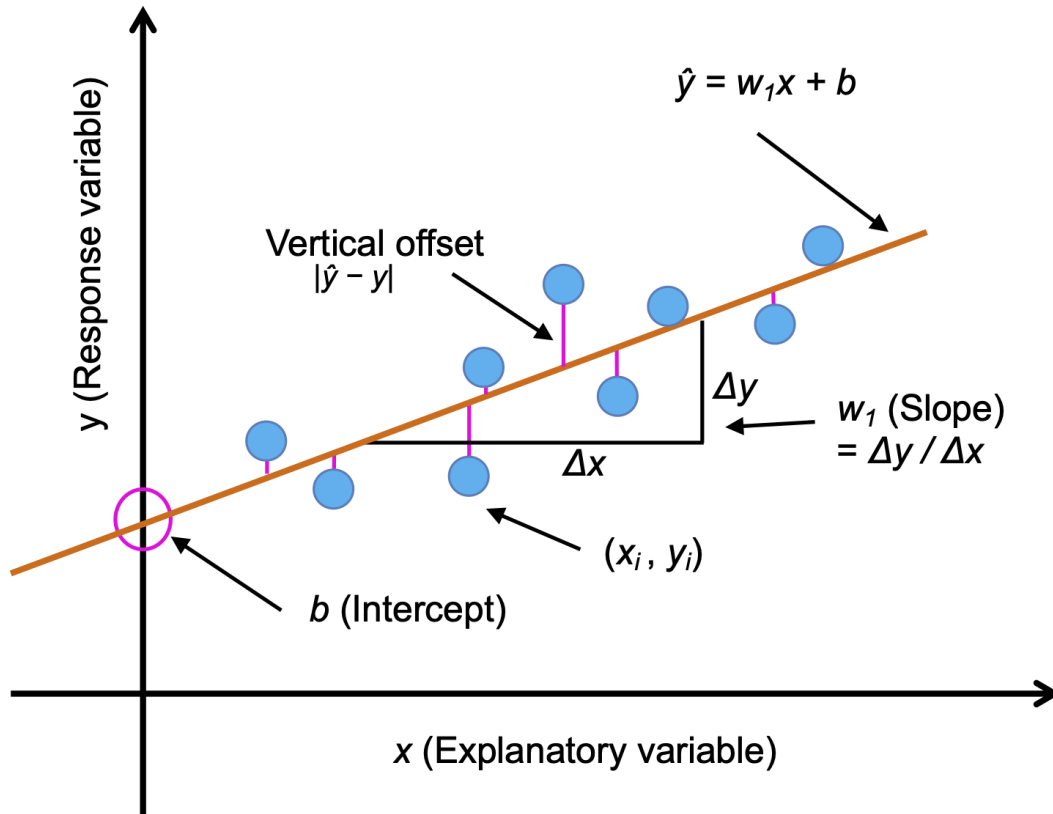






Regression

Simple (univariate) Linear Regression – Concept

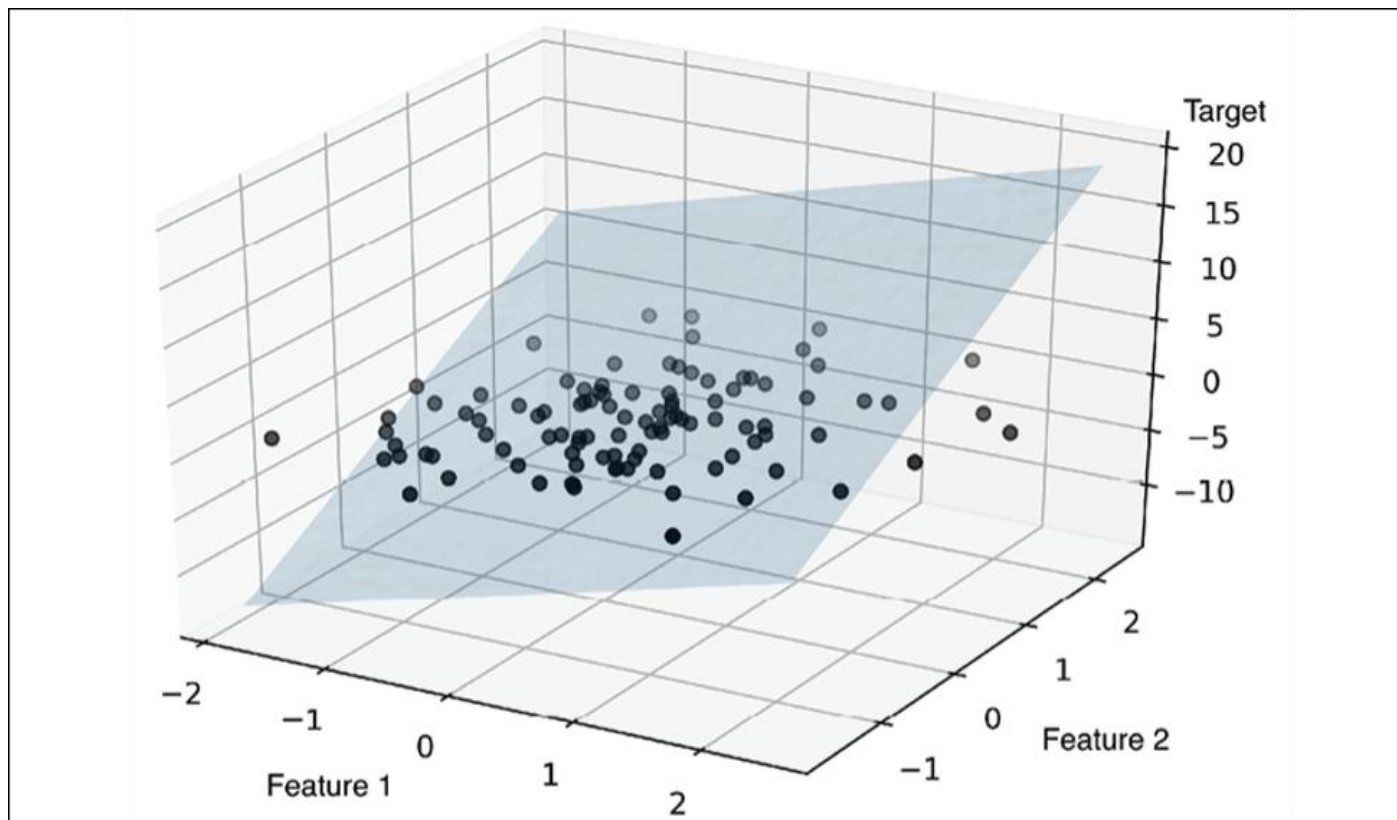


- Minimization equation:

$$\text{RSS} = \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2$$

- RSS: Residual Sum-of-Squares
- n = number of observations
- p = number of (input) features
 - Univariate regression: $p = 1$
 - Multivariate regression: $p > 1$

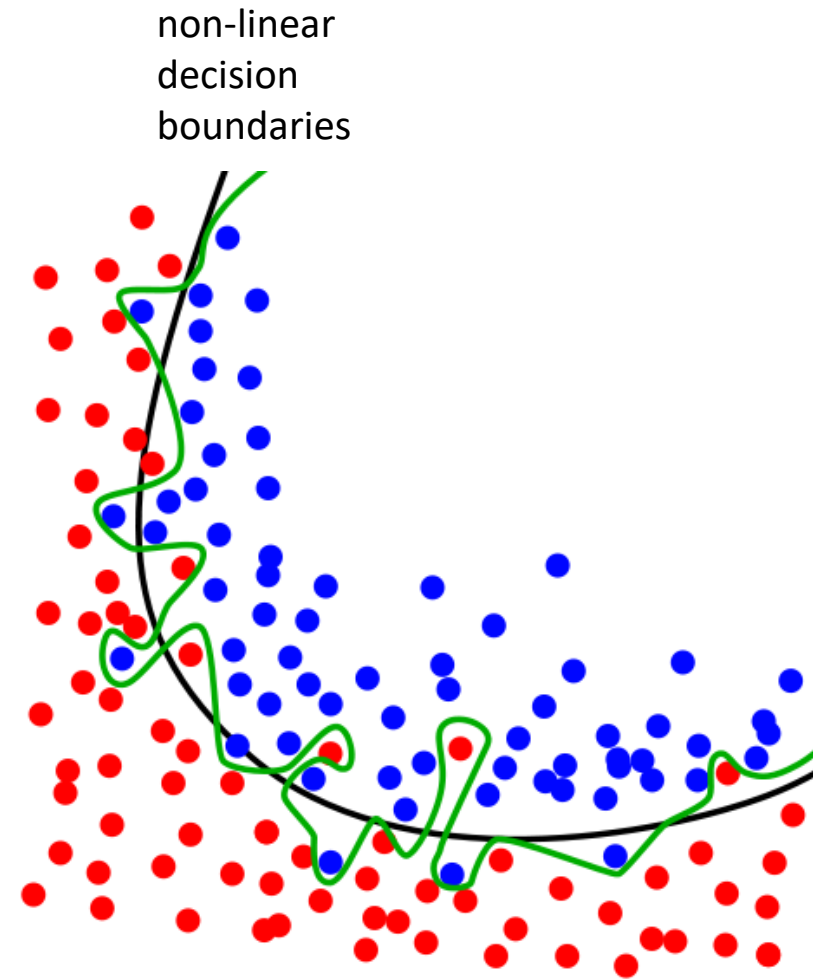
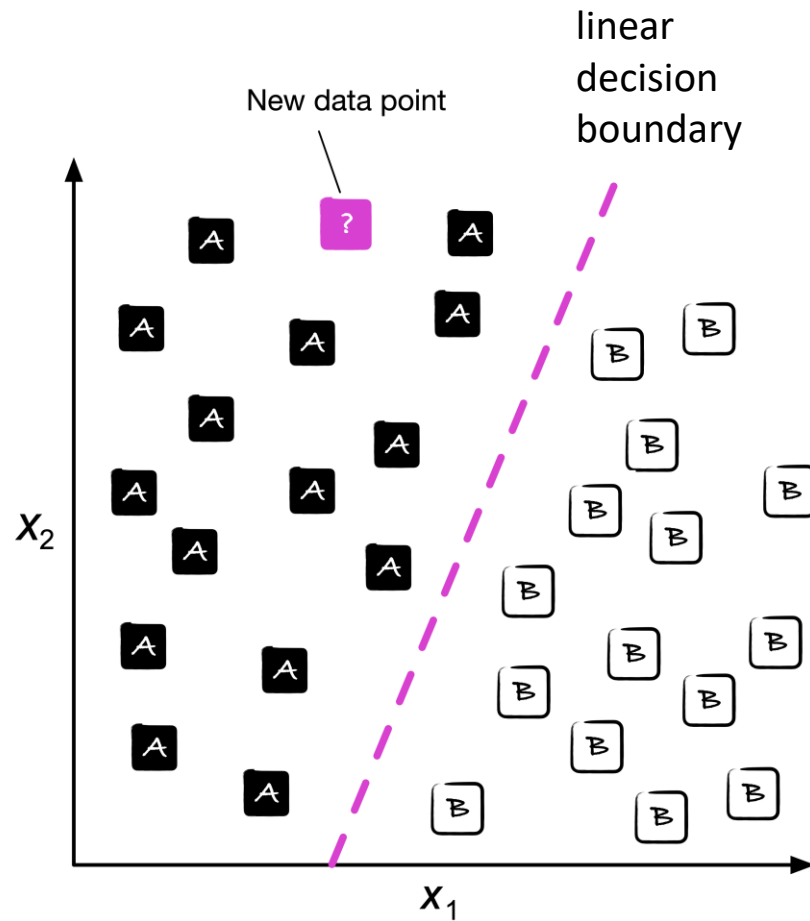
Multiple Linear Regression – Idea



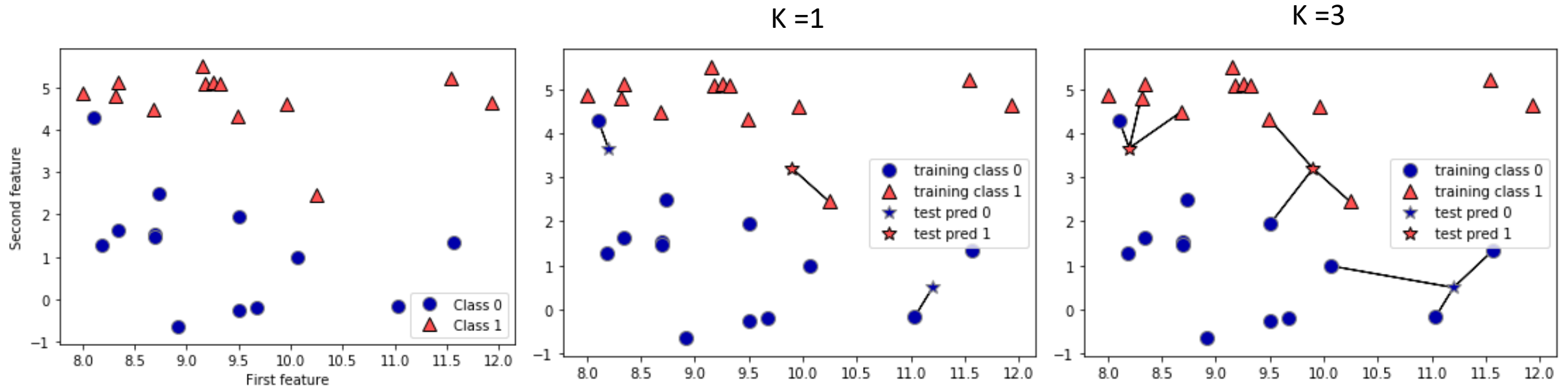
- $p = 2$: best fit is a plane
 - $p > 2$: best fit is a hyperplane
 - p can be tens/hundreds/thousands in common ML problems
- (**Q:** *are all features informative?*)

- **MSE** (mean squared error): $\frac{1}{n} \sum_{i=1}^n (y_i - \hat{f}(x_i))^2$
(y_i are actual target values, $\hat{f}(x_i)$ are predicted target values)
- **RMSE** (root mean squared error)
 - Square root of the above equation
 - Advantage: Expressed in same units as y
- **MAE** (mean absolute error): $\frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$ ($\hat{y}_i$ are predicted target values)
 - Large errors are less penalized than in MSE
 - Also expressed in same units as y

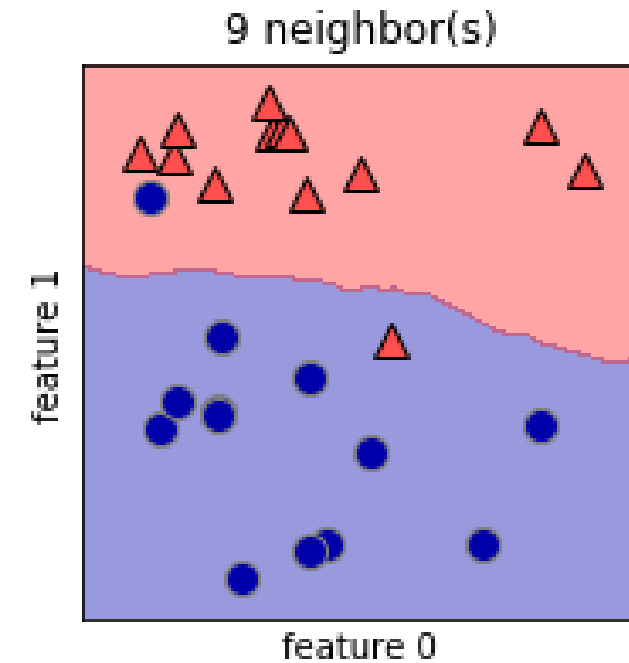
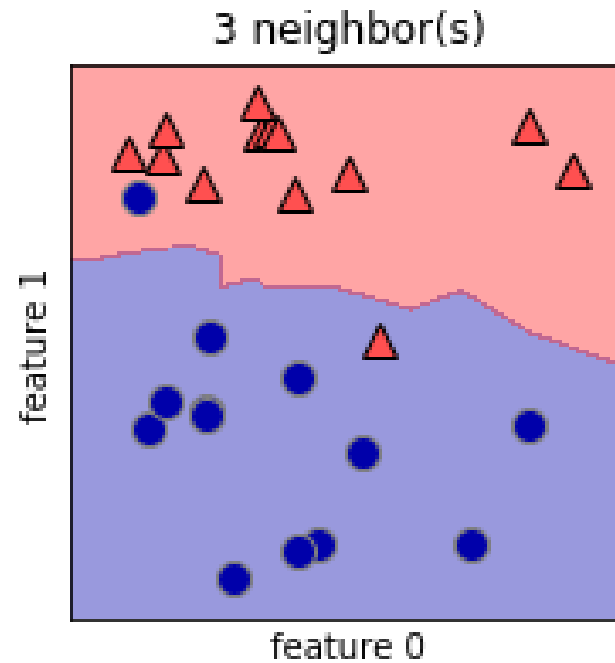
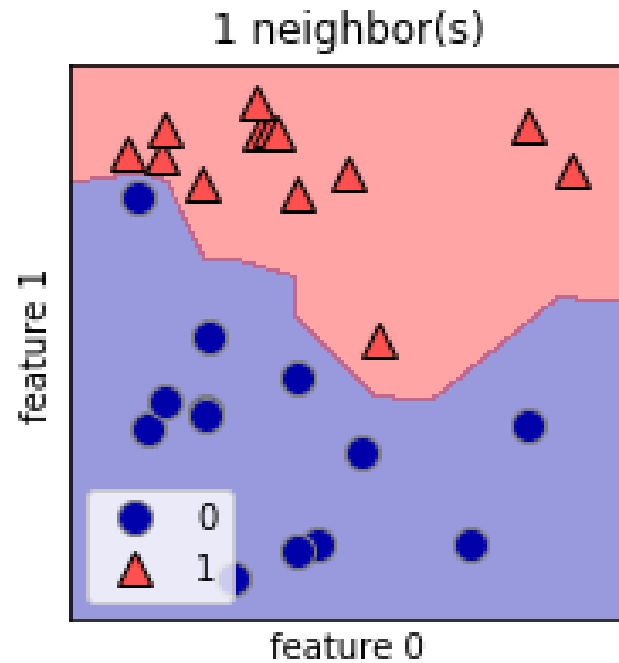
Classification

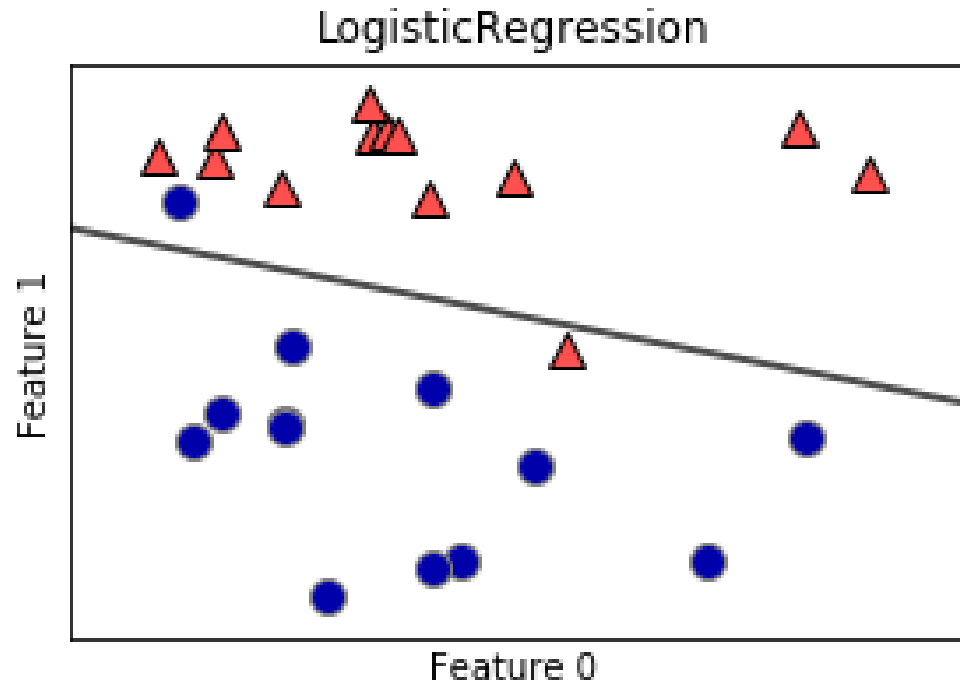
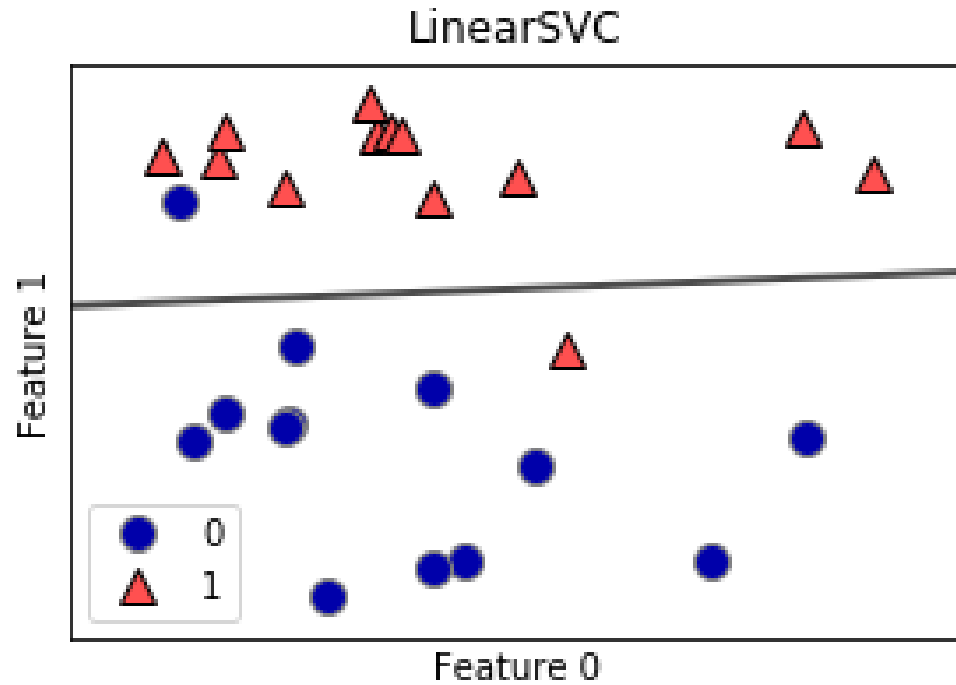


A simple classification algorithm – K-nearest neighbor



K-nearest neighbor decision boundaries





How to decide which algorithm (incl. hyperparameters) will give the best performance on new datapoints?
→ Model Evaluation

Classification – Metrics (using the Confusion Matrix)

		ACTUAL	
		Positive	Negative
PREDICTED	Positive	True Positive (TP) 5	False Positive (FP) 10
	Negative	False Negative (FN) 15	True Negative (TN) 70

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = 0.75$$

$$\text{Recall} = \frac{TP}{TP + FN} = 0.25$$

$$\text{Precision} = \frac{TP}{TP + FP} = 0.33$$

$$\text{F1 Score} = \frac{2 * \text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}} = 0.28$$

- When judged from the training data only, the performance of a model tends to be too optimistic (e.g., errors too low)
- A more realistic indicator for how well a model will predict new (future) data points is obtained when an independent test set is used to assess performance
 - Independent means here: not used in any way in the training step
- Common strategy: Split the data into training and test sets
 - Single split, e.g. 80:20 training:test sets (**not recommended on small datasets**)
 - Multiple single splits with different shuffles of the data (*Monte Carlo cross-validation*)
 - *K-fold cross-validation* (usually 5- or 10-fold, sometimes leave-one-out)
 - Multiple splitting methods: Random, stratified, group splits, etc.

Model evaluation – K-fold cross-validation

